

Assignment on Bash

Release day: 16th May

Submission on: 26th May

In the questions that follow, if you are required to write a bash script named `script.sh` with example arguments `<url>`, `<inputfile>` and `<outputfile>`, a wrong usage of the script should be flagged with the message below and exit with status 1.

Usage: `./script.sh <url> <inputfile> <outputfile>`

Web mining is essentially the process of using automated tools to find hidden patterns and useful information on the internet. Instead of humans manually digging through pages, these tools "sift" through websites and server records to turn messy data into clear insights.

Subtasks

1. (Hint: `wget`, `grep`, `pdftotext`) The Portable Document Format (PDF) is a file format developed in the 1990s to represent and exchange documents. A PDF ensures that layout, fonts, and images remain exactly the same when opened on any device, making it ideal for printing and sharing. While PDF documents are meant to be read by humans, sometimes we need to extract information from a PDF using a program. Today we'll learn how to count the number occurrences of a given word in a PDF. Your goal is to write a **bash** script to get the pdf from an online link and count the number of words in it.

Write a script, `pdfWordCounter.sh` (Usage: `./pdfWordCounter.sh <URL> <Word>`) which:

- a. Downloads the PDF from URL.
- b. Converts the PDF into text and displays the count of `<Word>` using `pdftotext` and `grep`. The match should be case-insensitive and whole-word only; for instance, `'break'` will match `'Break'`, and even the `'break'` in `'break.'` but not match `'breakfast'`.
- c. Deletes the pdf file that was generated
- d. **Note** that your script should only generate one pdf file during execution and then delete it on exit. A text file should not be created even temporarily while doing **b** step. Only output on the terminal should be the number of occurrences of the given word

Use: [this link](#) as an example URL and `'code'` as an example word.

[10 marks]

2. Write a script `downloadContents.sh` that performs the following tasks

Usage: `./downloadContents.sh <url> <directory_path>`

- a. Given a URL as the first argument, recursively download the web pages and files reachable from it, but restrict the crawl to: (i) the same domain as the starting URL, and (ii) directories at the same level as, or below, the directory containing the starting URL. For example, starting from `facebook.com/A/B/C/page.html`, the script **should not** crawl `Spicejet.com`, **nor should it** crawl parent-directory pages such as `facebook.com/A/B/ad.html` or `facebook.com/A/index.html`. However, all files needed to display the downloaded HTML pages, such as CSS files and images, should also be downloaded, even if they are located outside the starting directory, provided they are still on the same domain. In addition, convert all links in the html files to point to files you have just downloaded. Download the files in a folder named by the second argument to the script. Use suitable `wget` options to enforce all these restrictions.

(For more on `wget`, see [here](#))

[5 marks]

- b. Use the `tree` command to store the directories content in JSON format (read more about JSON [here](#)). Store this object in a file called `urlReport.json` [2 marks]
- c. Print the `md5sum` of this newly made file. [2 marks]
- d. Find the number of '{' (open curly braces) in this file and print this on a new line. Use `tr`.

[1 mark]

- e. Call the number obtained in d. as `n`. If there is a process with PID `n`, print the name of this process (as mentioned in the `COMMAND` column of `ps aux` output), else if there's no such process print "No such process" without quotes (print on new line). Use the `ps` command (with the write arguments). `ps aux` for e.g. lists all processes in your system.

[5 marks]

Output Format : (everything on a new line and no spaces anywhere), Example below:

Your output	
<code>fa069666859c6576953fb9d0cf3e50eb</code>	<md5sum of the json file>
<code>59</code>	<number of { >
<code>migration/7</code>	<name of process> with PID 59

(Hints: `md5sum`, `grep`, `ps`, `tr`)

Use [this](#) and [this](#) as test URLs.

[Total: 15 marks]

3. **Deciphering a plot** (based on a true historical event)

(Useful commands: `tail`, `tr`, `grep`)

You have to write a shell script called `decipher.sh` with the following usage:

Usage: `./decipher.sh <url>`

What does `decipher.sh` do? Well, here is the story: It is 1586. Queen Elizabeth I reigns over England. You are Phelippes, a code decipherer who works for Sir Walsingham, the Queen's loyal spymaster. Sir Walsingham suspects that Mary, Queen of Scots is scheming against Queen Elizabeth, and has her imprisoned her Chartley Castle. You are posted there in order to unravel the plot.

Your job is to intercept Mary's correspondence and look for incriminating evidence. This isn't so straightforward--Mary encrypts her letters! Transported to 1586 but armed with your laptop running Unix, you have to write a shell script `decipher.sh` to do the following:

- a. Download the following [url](#) and save it as `encrypted.txt` **[2 Marks]**
For convenience, use the quiet mode to ensure `wget` does not print anything to the terminal.
- b. You know this is a letter that Mary has written! It is encrypted using a rather simple **Caesar shift cipher** eg. if $A \rightarrow G$ then $B \rightarrow H$, $C \rightarrow I$, ..., $Z \rightarrow F$ (This is an oversimplification. In reality, a Caesar cipher, which just shifts alphabet letters, would have been far too easy for Elizabethan codebreakers like Thomas Phelippes to crack. You know that the last line of the letter has her signature block. Try all possible shifts for the last line of `encrypted.txt`, and check if the deciphered text contains "Queen" or "Majesty" or "Marie" or "Mary". (Hint: the `tail`, `tr` and `grep` commands should be useful. Your `grep` search should be case-insensitive) **[6 Marks]**
- c. Having figured out the key to the cipher, decipher the whole letter and save it in `deciphered.txt` If you've done it right, this should be English we can understand. Make sure that you preserve the formatting in terms of newlines. We expect the use of the `tr` command. **[2 Marks]**
- d. Indeed, Mary is guilty of high treason. But, the wily fox that you are, you must nab all the conspirators. It's time to use the broken cipher to your advantage! Using the key to the cipher, encrypt the line "I would be glad to know the names and qualities of the six gentlemen which are to accomplish the designment." (without quotes) and append the encrypted line immediately after the last line of `encrypted.txt`. **[2 Marks]**
You have to submit `decipher.sh`, we will generate (the modified) `encrypted.txt` and `deciphered.txt` when we run your script.

[Total: 12 Marks]

4. **Make An Autograder! (55 Marks)** For those of you who want a pointer to quickly get started with bash scripting, [this is nice](#). Also listen to the lectures on shell scripting.

Your code for this task will also be evaluated by a script, so please be careful and follow the naming conventions we specify. Automated grading is something you'll see more than once, and hopefully, doing this assignment will really *show* you why everyone insists that you follow the submission guidelines faithfully.

So here's the scene. You're a TA for a lab course. There has been an assignment. You've been assigned the task of verifying that the code people have turned in indeed gives the correct results, and awarding credit accordingly. You have to write three shell scripts for this task: `download.sh`, `organise.sh` and `evaluate.sh`. These scripts will be under the same directory.

As for awarding *you* credit for your scripting, *marks for each of these parts will be awarded independently*. That is, you can assume that while we grade your work, your `organise.sh` will see the correct state from `download.sh`, and so on. We take a small example to explain the directory structure at each step (you can use the `tree` command to verify). We start with the following empty files that you will fill in eventually.

```
•
|-- download.sh
|-- evaluate.sh
`-- organise.sh
```

a. `download.sh` (Useful commands: `wget`, `basename`)

[10 Marks]

To start, you've been given a link like [this](#) (when you write this link, don't forget the slash at the end), from where you can pull the documents you need. You need to write a script, `download.sh`, that is invoked as follows:

Usage: `./download.sh <link to directory> <cut-dirs argument>`

On running as intended, this should save the online directory locally as `mock_grading`. This directory will be under the same directory as that of the scripts. The local directory should have the *same structure* as the online one, the only difference being that the local version should not have the `index.html` files (`wget` does have an option of "rejecting" specified files). You may still have unwanted files, with the `.tmp` extension, which should be rejected using the above option.

Use `wget` in the quiet mode for this task. It is clear that you want to use the recursive option, and you do not want to download stuff from parent directories. If you don't specify anything further, you'll see that the downloaded directory you actually want is nested in directories, starting from the one corresponding to the host, eg.

`www.cse.iitb.ac.in`, within that `~as`, within that `bash-assignment` and finally within *that* is the directory you actually need.

`wget` has an option for not creating a directory corresponding to the host, but even if you invoke that, you're still stuck with 2 levels of nesting that you don't really want. That is where the `cut-dirs` flag will help. You don't need to worry about your script being fed the wrong argument for this.

Finally, your script needs to make sure that the local directory is called `mock_grading`. The online directory may have a different name. Check if that's the case (`basename` might do the trick), and if so, rename the directory you just got locally.

At this point, the directory structure should look something like (assume that `inputs`, `outputs`, `submissions` do not have any further subdirectories):

```
.
|-- download.sh
|-- evaluate.sh
|-- mock_grading
|   |-- inputs
|   |   |-- 0.in
|   |   |-- 1.in
|   |-- outputs
|   |   |-- 0.out
|   |   |-- 1.out
|   |-- roll_list
|   |-- submissions
|       |-- 180070026bar.cpp
|       |-- 180070035.cpp
|       |-- 180070035foo.txt
`-- organise.sh
```

Note: The `cut-dirs` parameter will be specified in such a way that just the `wget` command will save the downloaded directory locally with the same name as the remote one. In this example, it will be 2.

b. **organise.sh (In, for)**

[20 Marks]

Alright! You now have the roll list of students, the stuff they have turned in, the inputs to test their code, and the expected output corresponding to each input.

There's a small problem though. If you look in the `submissions` directory, you'll find that everyone's files are together. If you run code from here, it might just so happen that Bob's `.cpp` file finds Alice's `.h` file particularly helpful. Thankfully, the files are named honestly: every file begins with the roll number of the student who submitted it, and we assume that all roll numbers are of equal length.

Our `organise.sh` will be called as follows:

Usage: `organise.sh`

On running, it should create a directory called **organised** under the same directory as the scripts. Within the **organised** directory, it should create directories named after every roll number, eg. **180070026/**, **180070035/**, ... (see **roll_list** under the **mock_grading** directory for the complete list of roll numbers)

You might have guessed where this is going. However, we are not going to copy paste files from **mock_grading/submissions** into these roll number directories. Some of these files may be large, and this might waste space. What we'll do instead is simply create symbolic links (**ln -s**) to each of the files that the student has submitted.

For example, if roll number 180070035 has submitted **180070035.cpp**, then in **organised/180070035**, the file **180070035.cpp** is a symbolic link that'll point to **../../mock_grading/submissions/180070035.cpp**

Make sure you use relative paths while creating the symbolic links.

Now, the directory structure should look like:

```
.
|-- download.sh
|-- evaluate.sh
|-- mock_grading
|   |-- inputs
|   |   |-- 0.in
|   |   |-- 1.in
|   |-- outputs
|   |   |-- 0.out
|   |   |-- 1.out
|   |-- roll_list
|   |-- submissions
|       |-- 180070026bar.cpp
|       |-- 180070035.cpp
|       |-- 180070035foo.txt
|-- organise.sh
`-- organised
    |-- 180070026
    |   |-- 180070026bar.cpp -> ../../mock_grading/submissions/180070026bar.cpp
    |-- 180070035
    |   |-- 180070035.cpp -> ../../mock_grading/submissions/180070035.cpp
    |   |-- 180070035foo.txt -> ../../mock_grading/submissions/180070035foo.txt
```

For simplicity, you can assume that everyone who is on the roll-list has submitted at least one file and at most one **.cpp** file.

c. **evaluate.sh (g++, diff, timeout, redirection, sort)** **[25 Marks]**

We are now ready to do the actual grading. You can assume that each student has submitted at most one **.cpp** file; these are the only files we care about in this step. We will run this script as

Usage: evaluate.sh

On running, this script will generate two files: `marksheet.csv` and `distribution.txt`. These, as usual, will be in the same directory as our three scripts. `marksheet.csv` will have as many lines as there are students, `distribution.txt` will have as many lines as the number of test inputs.

`marksheet.csv` will have the score of students, sorted in lexicographical order of roll numbers. For example, a line could have: `180070035,2`

`distribution.txt`, on the other hand, will record for every input, how many students produced the correct output. For example, a line could have: `10, 6`

To calculate these scores, you need to do this for each student, from `organised/<roll no>`. First, compile the single `.cpp` source file into an executable called, wait for it, `executable`. The file may not exist, or may throw some compilation error, which causes `g++` to complain. You're a busy TA, you don't have time to deal with errors that aren't your own.

Suppress the error messages with `2>/dev/null` (2 refers to `stderr` which is, as it suggests, the standard file descriptor to which error messages are written, and what we do here is suppress stuff by redirecting it to a null device)

Now, the `inputs` and `outputs` in `mock_grading` come into play. For each input `mock_grading/inputs/foo.in`, the expected output is in `mock_grading/outputs/foo.out`

You need to run `executable` against each of the inputs, one by one, and store the generated output for `mock_grading/inputs/foo.in` in `organised/<roll no>/student_outputs/foo.out`

The nice thing here is that the `input` file will of course exist, and the `output` file will be generated regardless of whether `executable` actually exists (Why?)

But be careful! This is student code, there can be errors like segmentation faults, so you also need to redirect error messages to `/dev/null` just like you did while compiling. There's one more catch: you can't let code run forever, so set a `timeout` of 5 seconds for running each test case. For convenience, please also suppress segfault error messages by piping to the null command.

For each testcase, compare the generated output with the expected output. If they are **exactly** the same, add 1 to the student's score, else, do nothing. *Don't worry, we will only test you on cases where the expected output is not blank.*

Thus, you populate `marksheet.csv` as you evaluate each test case for each student. For `distribution.txt`, use an array whose size is the same as the number of inputs. If a student correctly produces the right output for input numbered `i`, add 1 to the count at index `i`.

Here's what your working directory will look like, finally: (in this instance, it so happened that `180070026bar.cpp` threw a compiler error) From the diagram, it should be clear that you're doing the compilation and testing from within `organised/<roll_no>` (Why?)

```
.
|-- distribution.txt
|-- download.sh
|-- evaluate.sh
|-- marksheet.csv
|-- mock_grading
|   |-- inputs
|   |   |-- 0.in
|   |   `-- 1.in
|   |-- outputs
|   |   |-- 0.out
|   |   `-- 1.out
|   |-- roll_list
|   `-- submissions
|       |-- 180070026bar.cpp
|       |-- 180070035.cpp
|       `-- 180070035foo.txt
|-- organise.sh
`-- organised
    |-- 180070026
    |   |-- 180070026bar.cpp -> ../../mock_grading/submissions/180070026bar.cpp
    |   `-- student_outputs
    |       |-- 0.out
    |       `-- 1.out
    `-- 180070035
        |-- 180070035.cpp -> ../../mock_grading/submissions/180070035.cpp
        |-- 180070035foo.txt -> ../../mock_grading/submissions/180070035foo.txt
        |-- executable
        `-- student_outputs
            |-- 0.out
            `-- 1.out
```

[Total: 55 Marks]

Acknowledgements: TAs of the 2019 offering of CS251. I was the instructor for the course.